

DEBRE BERHAN UNIVERSITY

COLLEGE OF COMPUTING

DEPARTMENT OF DATA SCIENCE

DEEPFAKE FACE DETECTION

Using Multi-Branch CNN with EfficientNetB3, DCT SRM

Debre Berhan, Ethiopia

Abstract

Deepfake generation techniques have advanced rapidly in recent years, creating serious concerns regarding the authenticity and reliability of digital media. As manipulated images and videos become increasingly realistic, conventional visual inspection methods are no longer sufficient for reliable detection. To address this challenge, this paper presents a multi-branch deep learning framework that combines three complementary facial representations: raw RGB appearance, Spatial Rich Model (SRM)-based residual noise patterns, and Discrete Cosine Transform (DCT)-based frequency information. By integrating both spatial and frequency-domain cues, the proposed system is able to capture semantic facial features as well as subtle manipulation artifacts introduced during face forgery.

The proposed architecture consists of three parallel branches. The RGB branch employs EfficientNetB3 as the backbone network for extracting high-level semantic features, while a custom four-layer convolutional neural network (CNN) is designed to learn SRM-based noise residual representations. In addition, a lightweight three-layer CNN is used to analyze DCT frequency maps and identify frequency-domain inconsistencies associated with manipulated content. Features extracted from the three branches are concatenated and passed through a multilayer fusion head for binary classification of real and fake faces.

Training is performed using a two-stage transfer learning strategy. In the first stage, the EfficientNetB3 backbone remains frozen while the fusion layers are optimized. In the second stage, the top 30 layers of EfficientNetB3 are fine-tuned using a lower learning rate to improve feature adaptation and generalization performance. Experimental results on the test dataset demonstrate that the proposed model achieves an accuracy of 82.94% and an AUC-ROC score of 92.12%, while maintaining a balanced precision and recall for both real and fake samples.

The results demonstrate that combining multiple visual cues significantly improves deepfake detection performance compared to relying on a single representation. Furthermore, the proposed framework provides a strong foundation for future extensions toward video-based deepfake detection and temporal attention modeling.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background of the Problem 1

1.2 Importance of This Project 1

1.3 Real-World Applications 2

1.4 Scope and Limitations 2

2 Problem Statement **2**

2.1 The Core Technical Problem 2

2.2 Reputation Destruction Through Fake Photos 3

2.3 Absence of Reliable Automated Detection 3

3 Objectives **3**

3.1 General Objective 3

3.2 Specific Objectives 4

4 Methodology **4**

4.1 Overview 4

4.2 Data Collection 4

4.3 Preprocessing Pipeline 5

4.4 Multi-Branch Network Architecture 9

4.5 Training Strategy 10

4.6 Evaluation Metrics 11

4.7 Model Architecture 11

4.8 Training Process 13

4.9 Tools and Technologies 14

5 System Design **14**

5.1 System Architecture Overview 14

5.2 System Block Diagram 15

5.3 System Flowchart 16

5.4 Three-Branch CNN Architecture 16

5.5 Processing Workflow 17

6 Implementation **17**

6.1 Implementation Modules 17

6.2 Core Functions 18

6.3 Model Architecture Summary 18

6.4 Sample Code Snippets 19

7 Results **20**

7.1 Training Performance 20
7.2 Final Test Performance 21
7.3 Confusion Matrix Analysis 21
7.4 Classification Report 22
7.5 Summary of Results 22

8 Conclusion **22**

References **24**

A Appendix A: Dataset Sample Visualization **24**

B Appendix B: Additional Model Results **25**

1. Introduction

1.1 Background of the Problem

Recently, advancements in artificial intelligence have enabled the creation of very lifelike images and videos, which are collectively called deepfakes. Deepfake videos are produced using deep learning algorithms, which are capable of swapping or modifying faces with great precision. Although this kind of technology has practical applications in sectors like entertainment and digital media creation, it poses several problems concerning misinformation, cyber fraud, privacy issues, and trust.

With the advancements in deepfake technology, there is a growing difficulty in distinguishing between real and altered content. Several of these fake videos are highly lifelike, which makes it harder for humans to determine them by visual analysis alone. Details on facial features, lighting conditions, and facial expressions are highly accurate, thus making conventional methods ineffective in their task.

Current deepfake detectors are mostly concerned with extracting facial features from RGB images. Despite this, there are still some invisible artifacts left in images after manipulation that could be used to detect them. They are usually present in image noise and frequency patterns created when the media is generated. Thus, considering just one type of visual feature might not suffice in all scenarios.In order to enhance the accuracy of detection, a novel multi-branch deep learning framework that integrates various facial feature spaces is proposed in this work. The model processes images in RGB format to recognize the visible image contents, SRM residual noise features to identify any underlying manipulation techniques, and DCT frequency maps to detect any discrepancies within the frequency domain.

1.2 Importance of This Project

Deepfake technology has gained increasing importance with the rising number of potential negative impacts in society due to fake content created by manipulation of digital media. Currently, deepfake technologies are sufficiently advanced to produce extremely realistic artificial images and videos. With time, this process will get easier and the use of deepfake media will increase significantly, leading to the spread of misinformation, reputation damage, and a lack of trust in social networks.

However, the misuse of deepfakes may result in multiple types of malicious behavior such as identity manipulation, online abuse, misinformation in politics, and digital scams. In addition, deepfake videos or images can be used as fabricated evidence or fake public statements which can influence legal investigations, opinion forming among people, and even national security. Consequently, manual analysis cannot provide an effective way to solve the problem of deepfake manipulation.

The majority of available solutions target face features detected by analyzing RGB data only. However, deepfake generation is gradually improving and producing more realistic deepfake images that do not reveal any visual manipulations. To overcome this obstacle, the present project uses multimodal forensics which combines RGB visual information with SRM noise patterns and DCT frequencies.

1.3 Real-World Applications

- **Social Media:** Pre-publication verification to detect any synthetic faces prior to publishing to avoid spreading them to a wider audience.
- **Law Enforcement:** Checking of photos or videos submitted for criminal or civil lawsuits.
- **Banks (KYC):** Detection of fake selfies that are being used by fraudsters to impersonate an existing bank account holder.
- **Journalism:** Authentication of received photos from the field in a matter of minutes when covering important events.
- **Government / Security:** Biometric detection to prevent the spread of synthetic biometric identities.

1.4 Scope and Limitations

The project addresses the problem of binary classification of face images from deepfakes, classifying them into real or fake categories. However, the project neither localizes altered parts of the image nor determines which GAN algorithm was used to generate the image. The project will exclude images in which the MTCNN does not detect any face.

2. Problem Statement

2.1 The Core Technical Problem

With the fast development of face synthesis techniques, it is easy for a computer to generate deepfake images that can fool people and simple detectors. Current solutions that focus on detecting visual artifacts or one specific modality will become less effective as the models continue developing to eliminate those artifacts. The fundamental question to be investigated in this study is:

> *"What kind of computer vision system can distinguish between genuine and deepfake face images by leveraging visual, noise, and frequency domain information?"*

Using a one-stream CNN on pixel values alone might not be able to detect some of the tampering effects that are undetectable in the visual domain but noticeable in the noise

and frequency domains. The question now becomes how best to develop an appropriate multistream system.

2.2 Reputation Destruction Through Fake Photos

One of the most damaging uses of the deepfake technology is its deployment as a means to tarnish the reputation of innocent people through the creation of a false face. It becomes easy to place an actual face on a doctored photograph of any kind; either placing that person in a compromising position or staging an imagined crime scene or attaching some fake social media content. The effects that result from these acts are irreversible since they have already been circulated before they are proved to be forgeries.

This is a form of digital defamation that is one of the major reasons for undertaking the project. In this case, the victim is likely a person of influence — an individual like a politician, journalist, businessman, or activist who suffers from defamation through the use of a fictitious image spread through social media channels. Nonetheless, regular private citizens are also susceptible as fictitious images are used for blackmail, romance frauds, and even whistle-blower silencing.

The issue is further complicated by the pace at which social media operates. A fake picture can reach millions of viewers within just a few hours after its release, much earlier than any fact-checking agency has time to react. With an automated detection system installed on social media uploading processes, one can prevent any harm from occurring.

2.3 Absence of Reliable Automated Detection

Even with the extent of damage inflicted by falsified face images, existing means of validating the authenticity of face images available to the general public, journalists, and law enforcement are either ineffective, unreliable, or unavailable. Forensic methods, which involve an extremely difficult process to be carried out by specialists, are also time-intensive. Current single-branch detectors are being continuously outsmarted by the newly emerging generative models. This work bridges this gap by designing and implementing an effective multi-branch detection system.

3. Objectives

3.1 General Objective

Design, development, and testing of a multi-branch deep learning architecture to detect deep fake face images with high precision using fusion of forensic cues extracted from RGB color space, noise residuals, and frequency domain.

3.2 Specific Objectives

- Gather and arrange a balanced database of genuine and synthetic faces based on public sources of deepfakes.
- Pre-process images by adjusting their size through aspect ratio-preserving padding, aligning faces via MTCNN, and extracting features in multiple modalities (SRM noise maps and DCT frequency maps).
- Propose a tri-branched CNN model that integrates EfficientNetB3-based transfer learning for RGB images with custom CNNs for the SRM and DCT branches.
- Model training will be done in two stages: Stage 1 involves training only the fusion head layer and keeping the rest frozen, while Stage 2 involves fine-tuning the top 30 EfficientNetB3 layers with a smaller learning rate.
- Utilize sophisticated generalization methods such as data augmentation, Test-time Augmentation, model ensemble, label smoothing, and class weight balancing.
- Assess model performance based on metrics like accuracy, AUC-ROC, confusion matrix, and per-class precision-recall.

4. Methodology

4.1 Overview

The proposed study presents a *multi-branch forensic deep learning model*, which integrates three distinct pieces of evidence, including visual information (RGB), residual noise-level information (SRM), and spectral domain data (DCT), to detect real images from fake ones. It is based on the idea that none of these cues individually is enough since GANs can generate very realistic-looking images in the pixel domain but exhibit some characteristics in the noise and frequency domains invisible to humans. The proposed approach will be discussed further.

$$\text{Image} \xrightarrow{\text{Resize+Pad}} \xrightarrow{\text{MTCNN Align}} \begin{cases} \text{RGB branch (EfficientNetB3)} \\ \text{SRM branch (4-layer CNN)} \\ \text{DCT branch (3-layer CNN)} \end{cases} \xrightarrow{\text{Fusion}} \hat{y} \in [0,1]$$

The following subsections describe each stage in detail.

4.2 Data Collection

The data collection process involved collecting images for creating the dataset through a combination of four open sources with benchmarks of deepfakes. All in all, the resulting

dataset consists of 19,000 facial images evenly divided into real and fake classes. This way, biases in training are avoided, ensuring that the model becomes generalized enough.

Real face images were primarily obtained from the dataset called Flickr-Faces-HQ (FFHQ), which can be found in the 140K Real and Fake Faces collection available on Kaggle. The mentioned dataset includes not only high-quality images of real people but also synthetic faces created using StyleGAN, allowing for a clear distinction between genuine and manipulated data.

To provide for diverse approaches to manipulations, we used Celeb-DF v2, in which images came from celebrities' faces swapped in deepfake videos. This will allow the model to see temporal deepfake artifacts in a frame-wise manner.

FaceForensics++ offered a variety of approaches to manipulating an image of a person's face, namely DeepFakes, Face2Face, FaceSwap, and NeuralTextures. Finally, the SFHQ was included since it offered more modern synthetic faces created with contemporary GAN and diffusion-based methods .All images were standardized into JPEG or PNG format and organized into two classes: real and fake. The dataset was then split using an 80/20 stratified approach, resulting in 3,776 test samples for final evaluation. Class balancing was applied during training to ensure stable learning across both categories.

Class-balanced training. The datasets used for training a deepfake detection algorithm in real life have an imbalance issue; hence, training with the unbalanced dataset will make the model favor the majority class. To avoid this bias issue, the weights associated with the classes are computed as:

$$w_c = \frac{N_{\text{total}}}{N_{\text{classes}} \times N_c} \tag{1}$$

where N_{total} is the number of samples in the training dataset, $N_{\text{classes}} = 2$, representing the classes, and N_c is the number of samples in class c. The weights are then provided to the `model.fit()` function to adjust the gradient contribution inversely proportionate to their frequency.

4.3 Preprocessing Pipeline

Images undergo a sequence of four pre-processing steps prior to being input into the network. The processing steps are outlined below along with their respective equations.

Step 1 — Aspect-Ratio Preserving Resize

Rationale. Square cropping results in distortions that could ruin forensic features in the image. The use of mean colour padding preserves the original aspect ratio.

Formulation. Every arbitrary-sized input image ($H \times W \times 3$) is resized such that the largest side corresponds to $T = 224$, with the smaller side centered on a canvas with fill color matching the spatial average of the image:

$$\text{scale} = \frac{T}{\max(H, W)}, \qquad H_{\text{new}} = \lfloor H \cdot \text{scale} \rfloor, \qquad W_{\text{new}} = \lfloor W \cdot \text{scale} \rfloor$$

$$\text{pad_top} = \left\lfloor \frac{T - H_{\text{new}}}{2} \right\rfloor, \qquad \text{pad_left} = \left\lfloor \frac{T - W_{\text{new}}}{2} \right\rfloor$$

$$\bar{\mathbf{c}} = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} I(i, j, \cdot) \quad \in \mathbb{R}^3$$

Canvas is initialized as $T \times T \times 3$ tensor having $\bar{\mathbf{c}}$, while the resized image is positioned from an offset of (pad_top, pad_left).

Why mean colour padding? Black padding causes a discontinuous, high contrast border that triggers edge sensitive neurons and may be misclassified as compression artifact by SRM noise extractor step in Step 3. Thus, mean color padding is employed to avoid such problems.

Step 2 — Face Alignment (MTCNN)

Definitions:

(Multi-Task Cascaded Convolutional Network) A cascaded network with three stages (P-Net → R-Net → O-Net) that simultaneously identifies faces and five facial keypoints (left eye, right eye, nose, left mouth, right mouth). It is used exclusively here for keypoint detection, with no face cropping taking place.

Transformation affine: A type of geometric transformation that includes rotation, uniform scaling, and translation. It is appropriate for facial normalization, as it maintains parallelism and straight lines.

Algorithm: MTCNN detects the pixel coordinates of the left eye $\text{LE} = (LE_x, LE_y)$ and right eye $\text{RE} = (RE_x, RE_y)$. A rotation–scale matrix is computed to bring both eyes to a horizontal baseline and normalise the inter-ocular distance to $T \times 0.3 = 67.2$ pixels:

$$\theta = \arctan 2(RE_y - LE_y,\ RE_x - LE_x) \quad \text{[rotation angle, radians]}$$

$$d = \sqrt{(RE_x - LE_x)^2 + (RE_y - LE_y)^2} \quad \text{[inter-ocular distance, pixels]}$$

$$s = \frac{T \times 0.3}{d} \quad \text{[scale factor]}$$

$$\mathbf{c} = \left(\frac{LE_x + RE_x}{2}, \frac{LE_y + RE_y}{2} \right) \quad \text{[rotation centre = eye midpoint]}$$

$$M = \texttt{cv2.getRotationMatrix2D}(\mathbf{c}, \theta, s) \quad \in \mathbb{R}^{2\times 3}$$

The aligned image is obtained by applying M via bilinear interpolation (`cv2.INTER_CUBIC`):

$$I_{\text{aligned}} = \texttt{warpAffine}(I, M, (T, T))$$

If MTCNN detects no face (e.g. heavily occluded or atypically posed image), the sample is discarded. All subsequent processing operates on I_{aligned}.

Step 3 —SRM Noise Residual Extraction

Spatial Rich Model (SRM) is a set of fixed high-pass filters originally developed for steganalysis. In this study, SRM is used to suppress semantic content in facial images and emphasize subtle noise patterns that are often associated with image manipulation. These noise patterns include sensor noise, compression artifacts, and synthetic generation fingerprints introduced by deepfake models.

The aligned RGB image $I_{aligned}$ is first converted into a grayscale representation $G \in [0,255]^{224\times 224}$ by removing color channels. This step ensures that the model focuses on intensity variations, where noise information is more prominent.

To extract residual features, three fixed 5×5 high-pass filters are applied using same padding to preserve spatial dimensions:

$$K_1 = \frac{1}{4}\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 2 & -4 & 2 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$K_2 = \frac{1}{12}\begin{bmatrix} -1 & 2 & -2 & 2 & -1 \\ 2 & -6 & 8 & -6 & 2 \\ -2 & 8 & -12 & 8 & -2 \\ 2 & -6 & 8 & -6 & 2 \\ -1 & 2 & -2 & 2 & -1 \end{bmatrix}$$

$$K_3 = \frac{1}{2}\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Each kernel extracts a different type of residual information. K_1 captures local second-order pixel dependencies, making it sensitive to small inconsistencies in texture. K_2 highlights complex high-frequency patterns often associated with GAN and diffusion-based generation artifacts. K_3 focuses on directional edge inconsistencies, which can reveal blending errors in face manipulation regions.

The residual maps are computed using 2D convolution:

$$N_n(i,j) = G(i,j) \circledast K_n$$

where $\circledast$ denotes convolution with same padding. Each resulting feature map is then normalized to the range $[0,255]$:

$$\hat{N}_n = \frac{N_n - \min(N_n)}{\max(N_n) - \min(N_n)} \times 255$$

The final SRM representation is constructed by stacking the three normalized residual maps:

$$SRM_{out} = \text{merge}[\hat{N}_1, \hat{N}_2, \hat{N}_3] \in [0,255]^{224\times224\times3}$$

The resulting image is stored in lossless PNG format to preserve exact residual information before being passed to the SRM branch of the network.

From a physical perspective, real images contain natural sensor noise introduced during image capture. In contrast, synthetic images generated by GANs or diffusion models lack true camera physics and instead introduce artificial patterns such as checkerboard artifacts, over-smoothed textures, and frequency inconsistencies. These differences become more pronounced in the SRM domain, allowing the model to learn discriminative forensic features effectively.

Step 4 –DCT Frequency Feature Extraction

The Discrete Cosine Transform (DCT) is a fundamental tool in image processing that represents an image in terms of spatial frequency components. It is widely used in JPEG compression, where it separates low-frequency structural information from high-frequency details. In deepfake detection, this property is useful because synthetic images often exhibit abnormal frequency distributions compared to real photographs.

Before applying DCT, the input image $I_{aligned}$ is converted into the YUV color space, and only the luminance channel $Y_f \in [0,255]$ is retained. This step is important because luminance carries most of the structural and intensity information, while chrominance channels are more

heavily compressed in typical image pipelines.

The two-dimensional DCT is then applied as:

$$F(u,v) = \frac{2}{N} \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} Y_f(i,j) \cos\left(\frac{\pi(2i+1)u}{2N}\right) \cos\left(\frac{\pi(2j+1)v}{2N}\right)$$

where $N = 224$ represents the image size and (u,v) correspond to spatial frequency indices.

To make the frequency representation more stable and suitable for learning, the magnitude spectrum is transformed using a logarithmic function:

$$L(u,v) = \log(|F(u,v)| + \varepsilon), \quad \varepsilon = 10^{-12}$$

This step reduces the large dynamic range of DCT coefficients and highlights subtle frequency variations that may otherwise be ignored.

The resulting values are then normalized into the range $[0,255]$:

$$DCT_{map}(u,v) = 255 \times \frac{L(u,v) - L_{min}}{L_{max} - L_{min}}$$

The final output is a single-channel image of size $224 \times 224 \times 1$, which is then fed into the DCT branch of the network for feature learning.

From a forensic perspective, real images tend to follow a natural frequency decay pattern approximately proportional to $1/f^2$. However, GAN and diffusion-based generators often violate this distribution due to the nature of upsampling and convolutional synthesis. This leads to unnatural frequency spikes and repeated artifacts across different scales. Additionally, JPEG compression introduces block-based distortions that are also visible in the DCT domain. The log-transformed DCT representation helps expose these inconsistencies, making them easier for the model to learn.

4.4 Multi-Branch Network Architecture

Motivation for multi-branch fusion. While each feature representation provides its own forensic signal, the model is actually created to combine all three representations. The RGB-based representation will learn semantics and textures, whereas the noise-based fingerprint of the SRM representation and the irregularities of the DCT representation will be used. Concatenated late fusion means that each channel can form its own representation, which can then be combined by the common classifier.

Branch 1 — RGB (EfficientNetB3). The RGB stem utilizes EfficientNetB3 that is pre-trained on ImageNet as a feature extractor. *EfficientNet* is a series of CNNs that scales width, depth, and resolution using a compound ratio, resulting in better accuracy per FLOP than networks that scale each parameter independently. During Phase 1, no weights are tuned except for the head, while in Phase 2, the top 30 layers of the EfficientNet backbone are unfrozen and fine-tuned with a lower learning rate (10^{-5}).

$$\mathbf{f}_{\text{rgb}} \in \mathbb{R}^{128} \quad [\text{output of RGB head, after two dense layers + dropout}]$$

Branch 2 — SRM (custom 4-layer CNN). A four-block convolutional network with $\{32, 64, 128, 256\}$ filters processes the three-channel SRM noise map. Each block is followed by batch normalisation and 2×2 max-pooling; global average pooling reduces spatial dimensions before a fully connected layer:

$$\mathbf{f}_{\text{srm}} \in \mathbb{R}^{128}$$

Branch 3 — DCT (custom 3-layer CNN). A three-block CNN with $\{32, 64, 128\}$ filters processes the single-channel DCT map. The shallower depth reflects the lower intrinsic dimensionality of the frequency feature:

$$\mathbf{f}_{\text{dct}} \in \mathbb{R}^{128}$$

Fusion classifier. The three branch embeddings are concatenated and passed through a three-layer MLP with ℓ_2 regularisation ($\lambda = 10^{-4}$) and progressive dropout:

$$\mathbf{z} = [\mathbf{f}_{\text{rgb}}; \mathbf{f}_{\text{srm}}; \mathbf{f}_{\text{dct}}] \in \mathbb{R}^{384}$$

$$\hat{y} = \sigma(W_4\,\text{ReLU}(W_3\,\text{ReLU}(W_2\,\text{ReLU}(W_1\mathbf{z} + b_1) + b_2) + b_3) + b_4)$$

where $\sigma(\cdot)$ is the sigmoid function, dimensions are $512 \to 256 \to 64 \to 1$, and $\hat{y} = 1$ denotes a *fake* prediction.

4.5 Training Strategy

Loss function. Binary cross-entropy with label smoothing ($\alpha = 0.1$) is used:

$$\mathscr{L}_{\text{BCE-LS}} = -\sum_i [(1-\alpha)\, y_i \log \hat{y}_i + \alpha\,(1-y_i)\log(1-\hat{y}_i)]$$

Label smoothing prevents the model from becoming over-confident and improves calibration by softening hard 0/1 targets to $(0.1, 0.9)$.

Two-phase training.

Phase 1 (head training): EfficientNetB3 backbone frozen; Adam optimiser, learning rate 3×10^{-4}; up to 40 epochs with early stopping (patience = 7, monitor = `val_auc`).

Phase 2 (fine-tuning): Top 30 backbone layers unfrozen; Adam optimiser, learning rate 10^{-5}; up to 30 epochs. Phase 2 only activates if Phase 1 achieves AUC ≥ 0.60.

Data augmentation. At training time the `ForensicGenerator` applies two stochastic augmentations consistently across all three modalities (RGB, SRM, DCT): (i) random horizontal flip ($p = 0.5$) and (ii) random brightness scaling by a factor sampled uniformly from $[0.85, 1.15]$. Augmentations are *not* applied at test time.

Inference: TTA + Ensemble. At test time, **Test-Time Augmentation (TTA)** averages five forward passes per sample (original + horizontal flip + three brightness variants) to reduce prediction variance. A weighted ensemble of the Phase-1 and Phase-2 checkpoints (0.4 and 0.6 respectively) further improves robustness. The final classification threshold is selected to maximise the Youden index $J = \text{TPR} - \text{FPR}$ on the ROC curve.

4.6 Evaluation Metrics

AUC-ROC (Area Under the Receiver Operating Characteristic curve): This is the main measure. A score of 1.0 represents perfection while 0.5 represents randomness. The AUC measure takes into consideration the discriminative power of a model for all thresholds.

Accuracy obtained at the best Youden index.

Confusion Matrix containing the number of true real, true fake, false alarms, and missed fakes.

Classification Report showing Precision

4.7 Model Architecture

The proposed system follows a three-branch multi-input convolutional neural network designed to learn complementary representations from RGB appearance, SRM residuals, and DCT frequency features. Each branch processes a different input modality and produces a 128-dimensional feature vector before fusion.

The forward propagation for each branch can be expressed as follows:

$$f_{rgb} = \text{Head}_{rgb}\left(\text{EfficientNetB3}(X_{rgb})\right), \quad X_{rgb} \in \mathbb{R}^{B\times 224\times 224\times 3}$$

$$f_{srm} = \text{Head}_{srm}\left(\text{SRM_CNN}(X_{srm})\right), \quad X_{srm} \in \mathbb{R}^{B\times 224\times 224\times 3}$$

$$f_{dct} = \text{Head}_{dct}\left(\text{DCT_CNN}(X_{dct})\right), \quad X_{dct} \in \mathbb{R}^{B\times 224\times 224\times 1}$$

The extracted feature vectors are concatenated into a unified representation:

$$f_{all} = \text{Concat}(f_{rgb}, f_{srm}, f_{dct}), \quad f_{all} \in \mathbb{R}^{B\times 384}$$

The final prediction is obtained through a fusion classifier with a sigmoid activation:

$$\hat{y} = \sigma\left(\text{FusionHead}(f_{all})\right), \quad \sigma(z) = \frac{1}{1+e^{-z}}$$

Convolutional Block Design

Each convolutional block in the network follows the general structure:

$$Z^{(l)} = \text{ReLU}\left(\text{BN}\left(W^{(l)} \circledast A^{(l-1)} + b^{(l)}\right)\right), \quad A^{(l)} = \text{MaxPool}(Z^{(l)})$$

Branch-wise Architecture

Branch 1 — RGB Stream: EfficientNetB3 (pre-trained on ImageNet, frozen during Phase 1) → Global Average Pooling → Dense(256, ReLU, L2 regularization) → Dense(128, ReLU) → Dropout(0.4). Output: 128-dimensional feature vector.

Branch 2 — SRM Stream: Conv2D(32) → Batch Normalization → MaxPooling → Conv2D(64) → Batch Normalization → MaxPooling → Conv2D(128) → MaxPooling → Conv2D(256) → Batch Normalization → Global Average Pooling → Dense(128, ReLU, L2) → Dropout(0.4). Output: 128-dimensional feature vector.

Note: The final Conv2D(256) layer directly uses Global Average Pooling without an additional pooling layer to preserve spatial detail before aggregation.

Branch 3 — DCT Stream: Conv2D(32) → Batch Normalization → MaxPooling → Conv2D(64) → Batch Normalization → MaxPooling → Conv2D(128) → Global Average Pooling → Dense(128, ReLU, L2) → Dropout(0.3). Output: 128-dimensional feature vector.

Fusion Head

The fused feature vector is processed through a fully connected classifier:

$$\text{Dense}(512, ReLU, L2) \rightarrow \text{Dropout}(0.5)$$
$$\rightarrow \text{Dense}(256, ReLU, L2) \rightarrow \text{Dropout}(0.4)$$
$$\rightarrow \text{Dense}(64, ReLU) \rightarrow \text{Dense}(1, \text{Sigmoid})$$

4.8 Training Process

The model is trained using Binary Cross-Entropy loss with label smoothing of 0.1 to reduce overconfident predictions and improve generalization. L2 regularization with $\lambda = 10^{-4}$ is applied to all dense layers in both branch-specific heads and the fusion head to reduce overfitting and improve stability during training.

Loss Function

Binary Cross-Entropy with label smoothing ($\varepsilon_s = 0.1$) replaces hard labels with soft labels $\{0.05, 0.95\}$:

$$y_{\text{smooth}} = y(1-\varepsilon_s) + \varepsilon_s/2$$
$$\mathscr{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_{\text{smooth},i}\log(\hat{y}_i) + (1-y_{\text{smooth},i})\log(1-\hat{y}_i)\right]$$

Optimizer — Adam

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla\mathscr{L}, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla\mathscr{L})^2$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta_t = \theta_{t-1} - \frac{\alpha\,\hat{m}_t}{\sqrt{\hat{v}_t}+\delta}$$

Phase 1: $\alpha = 3\times10^{-4}$; Phase 2: $\alpha = 1\times10^{-5}$; $\delta = 10^{-7}$.

Phase 1 — Head-Only Training

The EfficientNetB3 backbone is frozen (`trainable=False`). Training runs for up to 40 epochs with EarlyStopping (patience = 7, monitoring `val_AUC`), ModelCheckpoint, and ReduceLROnPlateau (factor = 0.3, patience = 4).

Phase 2 — Fine-Tuning (Conditional)

If Phase 1 achieves `val_AUC` ≥ 0.60, the top 30 layers of EfficientNetB3 are unfrozen. Training continues for up to 30 epochs at $\alpha = 1\times10^{-5}$ to prevent catastrophic forgetting.

Evaluation Metrics

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}, \quad \text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad \text{AUC-ROC} = \int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR})$$

$$t^* = \arg\max_t \left[\text{TPR}(t) - \text{FPR}(t)\right] \quad \text{(Youden's } J \text{ statistic)}$$

4.9 Tools and Technologies

Table 1: Tools and Libraries Used

Tool / Library	Purpose
Python 3.10	Main programming language used
TensorFlow 2.x / Keras	Model architecture, model training, and evaluation
OpenCV (cv2)	Loading images, converting color space, applying DCT, and affine transformation
MTCNN	Face detection and landmark alignment
NumPy	Array manipulation and feature map extraction
scikit-learn	Splitting data, calculating class weights, and metrics calculation
Matplotlib/Seaborn	Visualization of learning curves, confusion matrices, and ROC curve
PIL/Pillow	Reading images and saving them in lossless format as png files
Kaggle GPU Environment	Acceleration hardware for model training
tqdm	Progress tracking within pre-processing loops

5. System Design

5.1 System Architecture Overview

The proposed deepfake detection system follows a multi-stage pipeline that transforms raw face images into binary predictions (Real or Fake). The architecture integrates preprocessing, face alignment, forensic feature extraction, and multi-branch deep learning classification.

Table 2: System Pipeline Stages

Stage	Description
Input	Raw face images collected from `real/` and `fake/` directories.
Preprocessing	Resize images to 224×224 while preserving aspect ratio using padding.
Face Alignment	Detect facial landmarks with MTCNN and align faces to canonical orientation.
Feature Extraction	Generate three forensic representations: RGB image, SRM noise residual map, and DCT frequency map.
Classification	Three-branch CNN processes extracted features and fuses learned embeddings for final prediction.
Output	Binary classification: **Real** or **Fake** with probability score.

5.2 System Block Diagram

Figure 1 shows the major components of the detection pipeline and their connections.

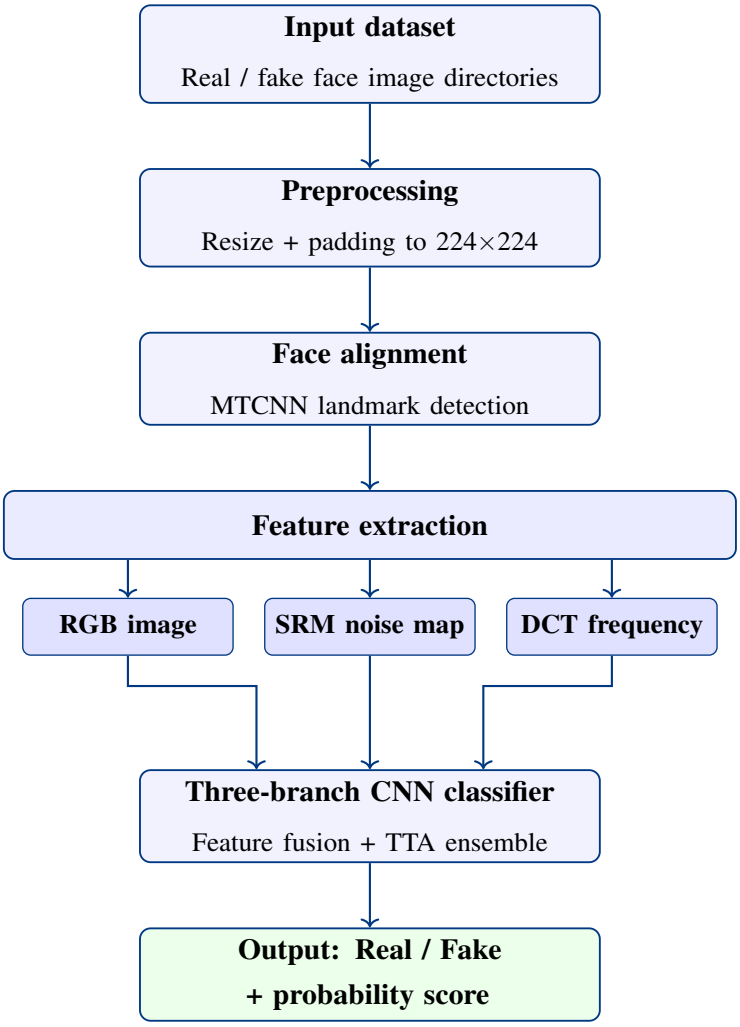


Figure 1: Block diagram of the deepfake detection system pipeline

5.3 System Flowchart

Figure 2 illustrates the step-by-step processing workflow of the proposed system.

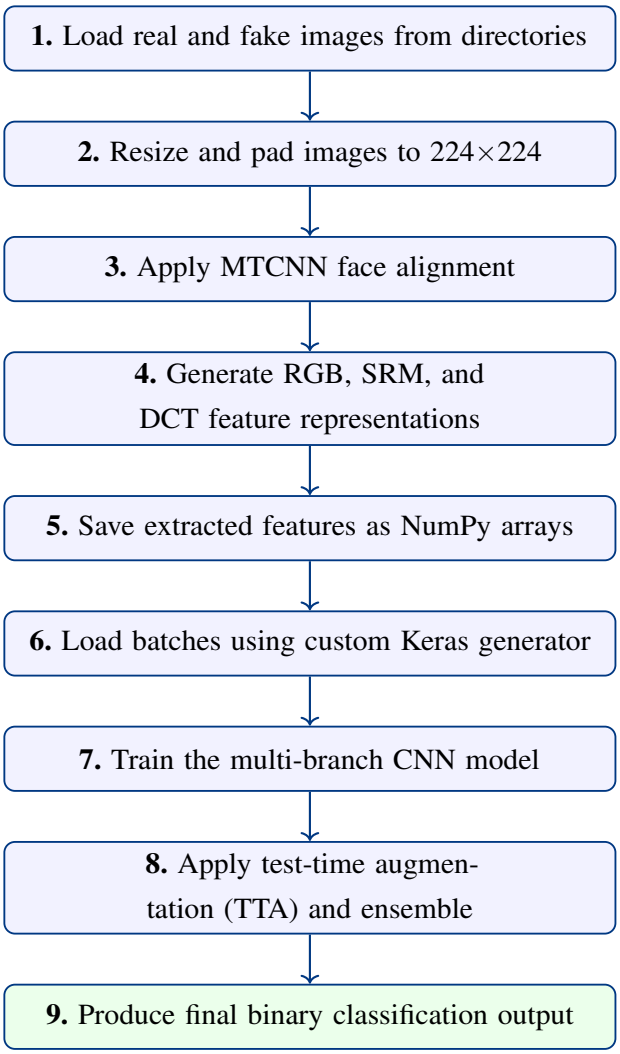


Figure 2: Processing flowchart of the proposed deepfake detection system

5.4 Three-Branch CNN Architecture

Figure 3 presents the input-to-output architecture of the proposed multi-branch CNN, showing how the three parallel streams are fused for binary classification.

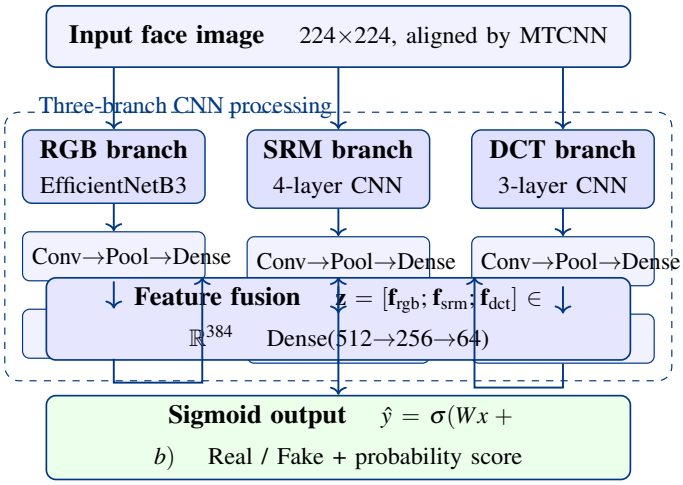


Figure 3: Three-branch CNN architecture: input → processing → output

5.5 Processing Workflow

The processing workflow consists of the following steps:

1. Load real and fake face images from dataset directories.
2. Resize and pad images to 224×224.
3. Apply MTCNN for facial landmark detection and alignment.
4. Generate RGB, SRM, and DCT feature representations.
5. Save extracted features as NumPy arrays.
6. Load batches using a custom Keras generator.
7. Train the multi-branch CNN model.
8. Apply test-time augmentation (TTA) and ensemble prediction.
9. Produce final binary classification output.

6. Implementation

6.1 Implementation Modules

The notebook implementation is divided into modular sections:

1. Dataset loading and exploration
2. Image preprocessing
3. Face alignment
4. Feature extraction (DCT and SRM)

5. NumPy dataset creation
6. Train-test split
7. Data generator creation
8. Model architecture definition
9. Phase 1 training
10. Phase 2 fine-tuning
11. Evaluation and visualization

6.2 Core Functions

Table 3: Main Functions Used in Implementation

Function	Purpose
`apply_resize_and_padding()`	Resizes images while preserving aspect ratio.
`align_face()`	Aligns detected faces using MTCNN landmarks.
`extract_dct()`	Extracts frequency-domain forensic features.
`apply_srm_filter()`	Generates SRM noise residual maps.
`ForensicGenerator`	Loads feature arrays with augmentation support.
`build_model()`	Constructs the three-branch CNN architecture.
`compile_phase2()`	Unfreezes EfficientNet layers for fine-tuning.
`predict_with_tta()`	Applies test-time augmentation inference.
`evaluate_model()`	Computes metrics, ROC, confusion matrix, and report.

6.3 Model Architecture Summary

The final model contains three input branches:

- **RGB Branch:** EfficientNetB3 pretrained on ImageNet.
- **SRM Branch:** CNN for noise residual analysis.
- **DCT Branch:** CNN for frequency-domain artifact detection.

Outputs from all branches are concatenated and passed through fully connected layers with dropout regularization before sigmoid classification.

$$P(y = 1|x) = \sigma(Wx + b) \tag{2}$$

where σ denotes the sigmoid activation function.

6.4 Sample Code Snippets

This section presents selected implementation snippets from the proposed deepfake detection pipeline.

Face Alignment using MTCNN

Face alignment was performed using MTCNN facial landmark detection. The left and right eye coordinates were used to estimate the rotation angle and align faces to a canonical orientation.

```
def align_face(img_rgb):
    results = detector.detect_faces(img_rgb)
    if not results:
        return None

    kp = results[0]["keypoints"]
    angle = compute_rotation(kp["left_eye"], kp["right_eye"])
    M = cv2.getRotationMatrix2D(center, angle, scale)
    return cv2.warpAffine(img_rgb, M, (224, 224))
```

DCT Feature Extraction

Frequency-domain features were extracted by applying the Discrete Cosine Transform (DCT) to the luminance channel.

```
def extract_dct(img_rgb):
    y = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2YUV)[:, :, 0]
    dct = cv2.dct(np.float32(y) / 255.0)
    return normalize(np.log(np.abs(dct) + 1e-12))
```

Three-Branch CNN Fusion Model

The proposed architecture combines RGB, SRM, and DCT branches before binary classification.

```
input_rgb = Input(shape=(224,224,3))
base = EfficientNetB3(include_top=False, weights="imagenet")

x_rgb = GlobalAveragePooling2D()(base.output)
merged = Concatenate()([x_rgb, x_srm, x_dct])

x = Dense(512, activation="relu")(merged)
```

```
output = Dense(1, activation="sigmoid")(x)
```

The above code snippets summarize the core implementation components of the proposed system, including preprocessing, forensic feature extraction, and multi-branch deep learning classification.

7. Results

7.1 Training Performance

The proposed deepfake detection model was trained in two phases using transfer learning with EfficientNetB3.

- **Phase 1:** Training only the custom classification head while keeping the EfficientNetB3 backbone frozen.
- **Phase 2:** Fine-tuning the top 30 layers of EfficientNetB3 with a lower learning rate.

The training history for both phases is shown below, including accuracy, loss, and AUC trends across epochs.

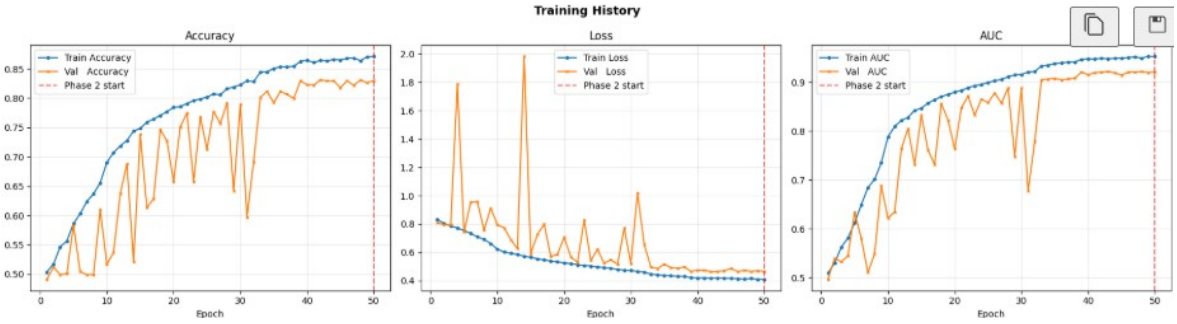


Figure 4: Training history of the proposed EfficientNetB3 model showing training and validation accuracy, loss, and AUC over 50 epochs. The red dashed vertical line indicates the start of Phase 2 fine-tuning.

From Figure 4, training accuracy steadily increased from approximately 49% to 86.8%, while validation accuracy stabilized near 83%. Training loss decreased consistently from 0.85 to 0.42, and validation loss converged near 0.47, indicating stable optimization with minimal overfitting. Validation AUC increased gradually and reached 0.9212.

Table 4: Performance Comparison Between Training Phases

Metric	Phase 1	Phase 2
Best Validation AUC	0.9202	0.9212
Best Validation Accuracy	0.8294	0.8310
Test Accuracy	0.8294	0.8294
Test AUC	0.9202	0.9212
AUC Gain	–	+0.0010

Phase 2 fine-tuning slightly improved validation AUC from 0.9202 to 0.9212, confirming that unfreezing higher EfficientNet layers improved feature adaptation for deepfake classification.

7.2 Final Test Performance

The best saved model (`model_phase2.keras`) was evaluated on the held-out test dataset.

Table 5: Final Test Performance Metrics

Metric	Value
Test Accuracy	82.94%
Test AUC-ROC	92.12%
Best Threshold	0.4191
Test Loss	0.4652

The final model achieved a test accuracy of 82.94% and an AUC score of 92.12%, demonstrating strong discrimination between real and fake images.

7.3 Confusion Matrix Analysis

Table 6: Confusion Matrix Results

	Predicted Real	Predicted Fake
Actual Real	1509	382
Actual Fake	241	1644

The model correctly classified 1509 real images and 1644 fake images. A total of 382 real images were incorrectly predicted as fake, while 241 fake images were missed.

7.4 Classification Report

Table 7: Classification Report

Class	Precision	Recall	F1-score	Support
Real	0.86	0.80	0.83	1891
Fake	0.81	0.87	0.84	1885
Accuracy			0.84	
Macro Avg	0.84	0.84	0.83	3776
Weighted Avg	0.84	0.84	0.83	3776

The classification report shows balanced performance across both classes. The Fake class achieved higher recall (0.87), indicating strong sensitivity for detecting manipulated images.

7.5 Summary of Results

Overall, the proposed model achieved:

- Test Accuracy of 82.94%
- AUC-ROC of 92.12%
- Balanced classification performance across both classes
- Improved validation AUC after fine-tuning in Phase 2

These findings confirm that the proposed EfficientNetB3-based framework is effective for deepfake image detection.

8. Conclusion

This project successfully designed and implemented a multi-branch deep learning system for detecting deepfake face images. By simultaneously analyzing three complementary forensic signals — raw RGB appearance via EfficientNetB3, noise residuals via SRM filtering, and frequency-domain anomalies via DCT — the system captures manipulation artifacts that are invisible to any single modality operating alone.

The two-phase transfer learning strategy balanced efficient use of pre-trained ImageNet knowledge with task-specific adaptation. Advanced generalization techniques, including

Test-Time Augmentation, weighted model ensembling, data augmentation, label smoothing, and class-weight balancing, were incorporated to improve robustness on unseen data.

All specific objectives were met: the data preprocessing pipeline transforms raw images into normalized, aligned, multi-modal representations; the model architecture follows a principled multi-branch fusion design; the training framework includes proper evaluation with AUC, confusion matrix, and per-class metrics; and the model identity verification confirms the correctness of the two-phase checkpoint system.

Future work could extend this system with video-temporal analysis (detecting temporal inconsistencies across frames), integration of attention mechanisms to focus on high-artifact regions, and deployment as a real-time web API for content moderation platforms.

References

Research Papers

[1] Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Niessner, M. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. *ICCV 2019*.

[2] Li, Y., Chang, M.C., & Lyu, S. (2018). In Ictu Oculi: Exposing AI Generated Fake Face Videos by Detecting Eye Blinking. *IEEE International Workshop on Information Forensics and Security*.

[3] Tan, M., & Le, Q.V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML 2019*.

[4] Fridrich, J., & Kodovsky, J. (2012). Rich Models for Steganalysis of Digital Images. *IEEE Transactions on Information Forensics and Security*, 7(3), 868–882.

[5] Zhang, K., Zhang, Z., Li, Z., & Qiao, Y. (2016). Joint Face Detection and Alignment using Multi-Task Cascaded Convolutional Networks (MTCNN). *IEEE Signal Processing Letters*.

Online Resources and Tools

- TensorFlow Documentation: `https://www.tensorflow.org/api_docs`
- Keras Applications — EfficientNet: `https://keras.io/api/applications/efficientnet/`
- OpenCV DCT Documentation: `https://docs.opencv.org/4.x/`
- MTCNN Python Package: `https://github.com/ipazc/mtcnn`
- Kaggle FaceForensics Dataset: `https://www.kaggle.com/datasets`
- Scikit-learn Metrics: `https://scikit-learn.org/stable/modules/classes.html`

Books

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Chollet, F. (2021). *Deep Learning with Python*, 2nd Edition. Manning Publications.
- Szeliski, R. (2022). *Computer Vision: Algorithms and Applications*, 2nd Edition. Springer.

A. Appendix A: Dataset Sample Visualization

This section shows how dataset samples were loaded and visualized during preprocessing.

```python
import os, cv2, random
import matplotlib.pyplot as plt

fake_path = "/kaggle/input/datasets/eso1844/final-model/fake"
real_path = "/kaggle/input/datasets/eso1844/final-model/real"

def plot_samples(path, title, n=4):
    imgs = random.sample(os.listdir(path), n)
    plt.figure(figsize=(12, 4))
    for i, name in enumerate(imgs, 1):
        img_path = os.path.join(path, name)
        img = cv2.imread(img_path)
        if img is None:
            continue
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        plt.subplot(1, n, i)
        plt.imshow(img)
        plt.title(f"{title}")
        plt.axis('off')
    plt.show()

for p, t in [(real_path, "REAL"), (fake_path, "FAKE")]:
    print(f"--- Inspecting {t} Samples ---")
    plot_samples(p, t)
```

[2] Python

```
--- Inspecting REAL Samples ---
```

Figure 5: Code used to load and display dataset images

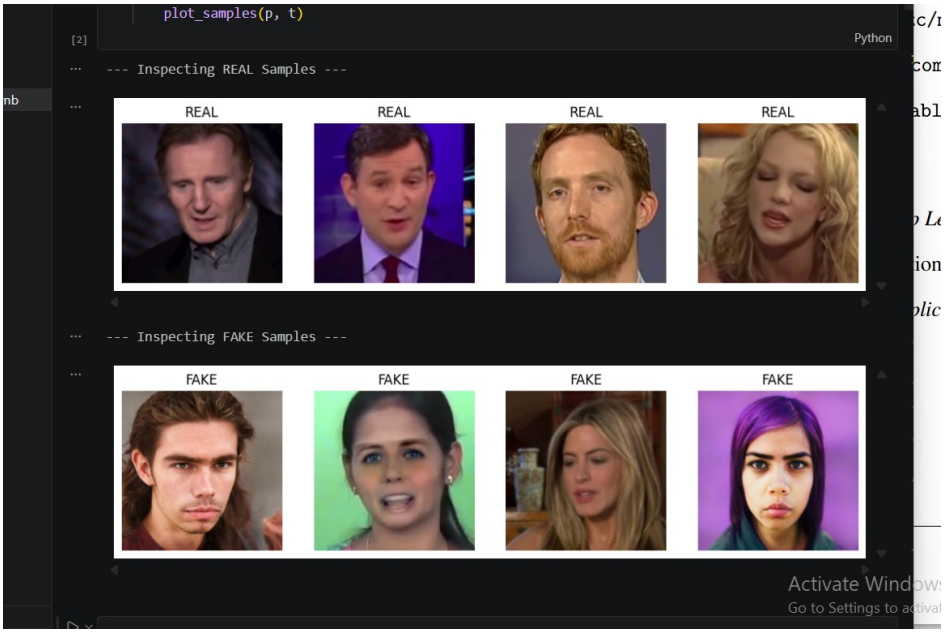


Figure 6: Sample visualization of real and fake face images

B. Appendix B: Additional Model Results

This section includes feature representations and evaluation results obtained from the proposed model.

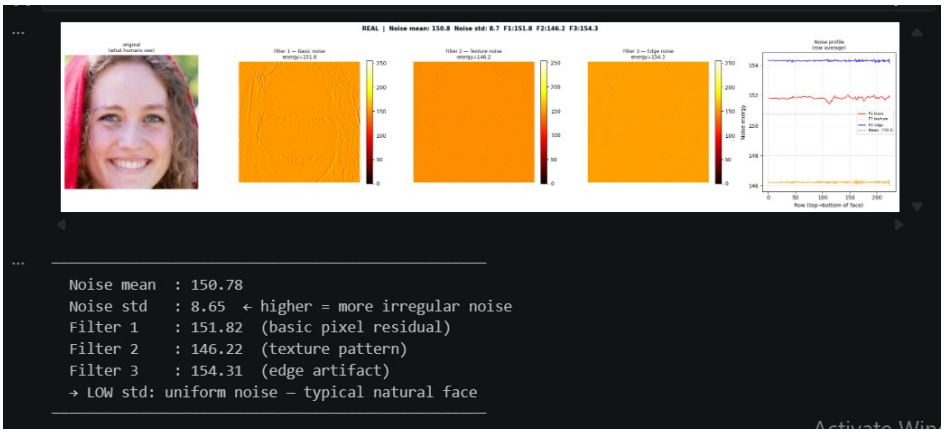


Figure 7: SRM noise residual feature map

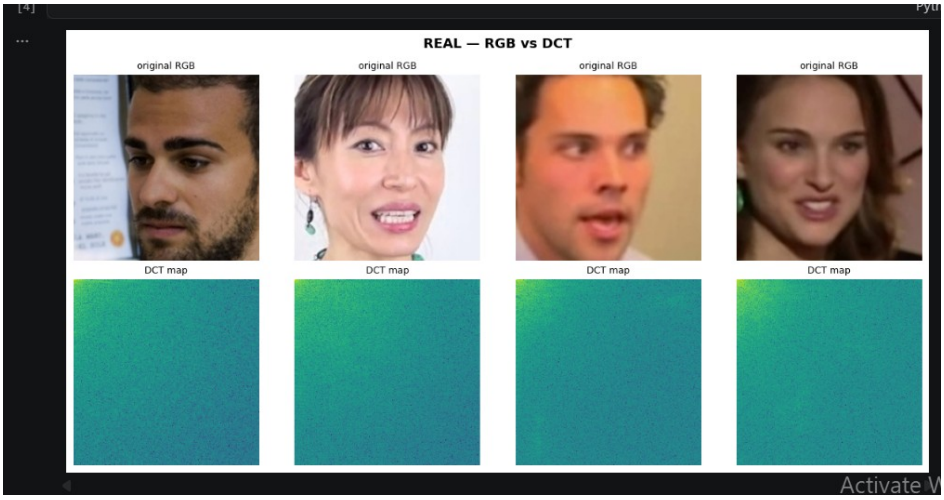


Figure 8: DCT frequency domain representation of input images

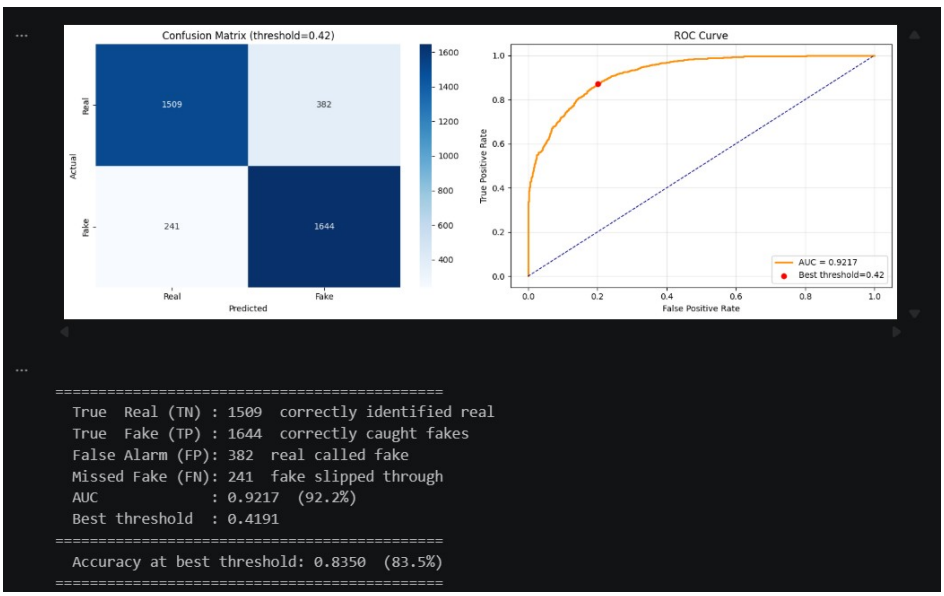


Figure 9: Confusion matrix showing classification performance

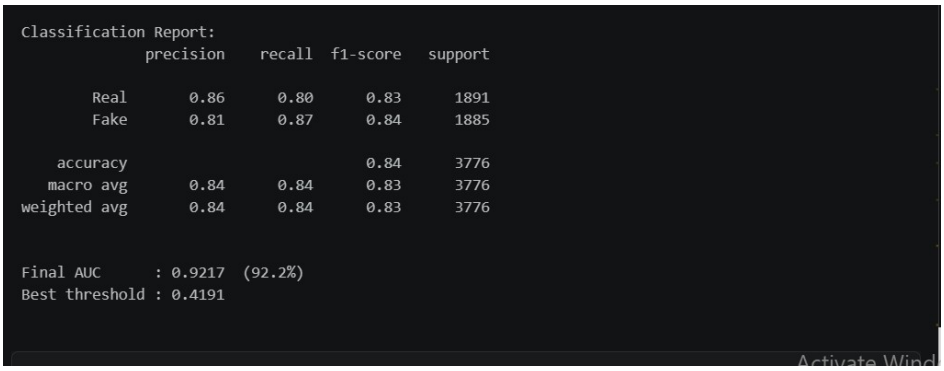


Figure 10: Classification Report: